

AuraMap

El Airbnb de las batallas de aura

Tema del proyecto, control de versiones, pruebas unitarias y dockerización de la aplicación.

Persona A · Release Manager **Rubén Coorahua**

Persona B · QA & Test Automation **Leonardo Candio**

Persona C · Platform / Container **José Osnayo**

Repositorio: github.com/AaronCoorahua/aura-map · Versión 1.1.0 · Septiembre 2026

Índice

a) Título y descripción del proyecto y del equipo

b) Justificación de la elección del tema

c) Aplicación de Git con Conventional Commits

d) Pruebas unitarias

e) Automatización en scripts bash

f) Dockerizar su aplicación

g) Conclusiones preliminares

Anexo: evidencias

a) Título y descripción del proyecto y del equipo

Nombre del producto: AuraMap — el "Airbnb de las batallas de aura".

"No es quién grita más fuerte, es quién pierde la compostura primero."

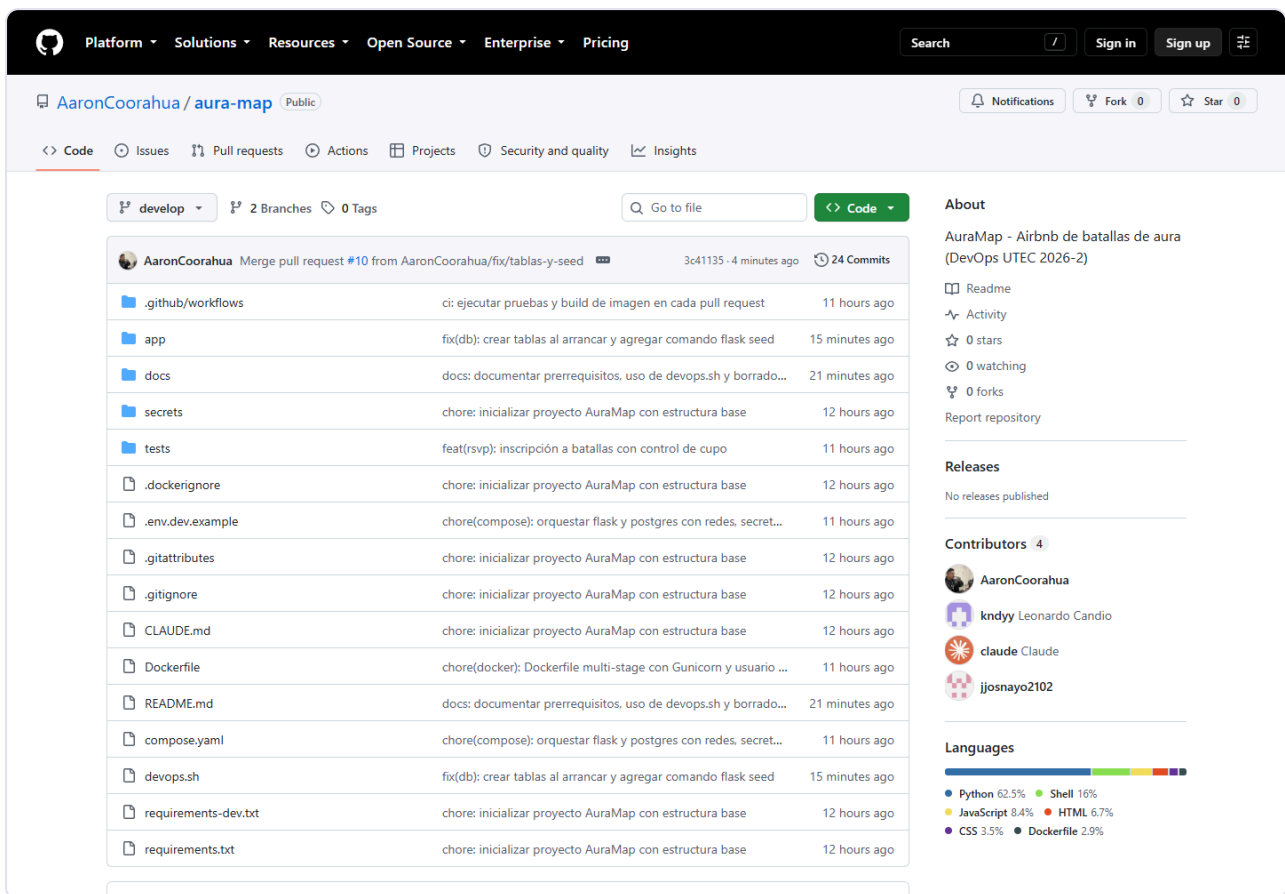
Problema y solución: organizar una batalla de aura hoy se coordina a mano (grupos de chat, historias de Instagram, TikToks con la ubicación en los comentarios, boca a boca), sin un lugar central donde ver qué batallas hay cerca, cuánto cupo queda o si tienen permiso municipal para no terminar desalojados por serenazgo a mitad del duelo. AuraMap resuelve esto con una app web mínima (sin login, sin pagos) donde:

- un organizador publica una batalla con lugar, fecha, cupo y si tiene permiso municipal;
- cualquiera busca batallas cercanas en un mapa (Leaflet + OpenStreetMap), filtra por distrito o por radio en kilómetros, y se inscribe con control de cupo y sin duplicarse.

Integrantes y roles:

Persona	Integrante (GitHub)	Rol DevOps	Responsabilidad principal
A	Rubén Coorahua (@AaronCoorahua)	Release Manager / Integración	GitFlow, protección de ramas, Conventional Commits, revisión de PRs, release y tag.
B	Leonardo Candio (@kndyy)	QA & Test Automation	Estrategia de pruebas, pytest y cobertura, pipeline de CI.
C	José Osnayo (@jjosnayo2102)	Platform / Container Engineer	Dockerfile, compose.yaml (redes, secretos, volumen, healthcheck) y devops.sh .

Los tres desarrollan features de producto además de su rol; el rol define de qué responde cada uno y qué revisa con más rigor en los Pull Requests.



The screenshot displays the GitHub repository page for **AaronCoorahua / aura-map**. The repository is public and has 24 commits. The main content area shows a list of files and folders with their commit history. The right sidebar contains information about the repository, including the README, activity, stars, forks, releases, contributors, and languages.

File/Folder	Commit Message	Time Ago
.github/workflows	ci: ejecutar pruebas y build de imagen en cada pull request	11 hours ago
app	fix(db): crear tablas al arrancar y agregar comando flask seed	15 minutes ago
docs	docs: documentar prerequisites, uso de devops.sh y borrado...	21 minutes ago
secrets	chore: inicializar proyecto AuraMap con estructura base	12 hours ago
tests	feat(rsvp): inscripción a batallas con control de cupo	11 hours ago
.dockerignore	chore: inicializar proyecto AuraMap con estructura base	12 hours ago
.env.dev.example	chore(compose): orquestar flask y postgres con redes, secret...	11 hours ago
.gitattributes	chore: inicializar proyecto AuraMap con estructura base	12 hours ago
.gitignore	chore: inicializar proyecto AuraMap con estructura base	12 hours ago
CLAUDE.md	chore: inicializar proyecto AuraMap con estructura base	12 hours ago
Dockerfile	chore(docker): Dockerfile multi-stage con Gunicorn y usuario ...	11 hours ago
README.md	docs: documentar prerequisites, uso de devops.sh y borrado...	21 minutes ago
compose.yaml	chore(compose): orquestar flask y postgres con redes, secret...	11 hours ago
devops.sh	fix(db): crear tablas al arrancar y agregar comando flask seed	15 minutes ago
requirements-dev.txt	chore: inicializar proyecto AuraMap con estructura base	12 hours ago
requirements.txt	chore: inicializar proyecto AuraMap con estructura base	12 hours ago

About
AuraMap - Airbnb de batallas de aura (DevOps UTEC 2026-2)
Readme
Activity
0 stars
0 watching
0 forks
Report repository

Releases
No releases published

Contributors 4
AaronCoorahua
kndyy Leonardo Candio
claude Claude
jjosnayo2102

Languages
Python 62.5%
Shell 16%
JavaScript 8.4%
HTML 6.7%
CSS 3.5%
Dockerfile 2.9%

Figura 1. Repositorio público en GitHub

b) Justificación de la elección del tema

El meme que se volvió plan de fin de semana. Desde 2024 el "aura" pasó de ser una palabra esotérica a la moneda social de la Gen Z y la Gen Alpha: en TikTok, Instagram y en cualquier salón de clases se reparten "+1000 de aura" por hacer algo con estilo y se quita "-500 de aura" por tropezarse frente a todos. En 2025 explotó el *aura farming* (acumular aura a propósito con pose, calma y actitud de protagonista de anime), y lo que empezó como comentarios en videos se convirtió en algo presencial: grupos de jóvenes que se juntan en un parque o una plaza para medirse en **batallas de aura** — duelos de presencia, estilo y compostura donde el público decide quién se lleva los puntos.

El problema real detrás del meme. Hoy esas batallas se organizan con un flyer en historias que dura 24 horas, un grupo de WhatsApp que explota y una dirección pasada por DM. Pasa siempre lo mismo: los que *quieren participar* no saben si queda cupo, los que solo *quieren ir a ver* no se enteran a tiempo o no saben si les queda cerca, y los organizadores no pueden decir si tienen permiso municipal (clave para que serenazgo no corte el evento). Es la misma necesidad que resolvió Airbnb con los alojamientos: un mapa, una ficha clara y un botón para reservar tu lugar.

Por qué este tema para un curso de DevOps. El curso pide construir un proyecto real con un flujo DevOps completo (Git con convenciones, pruebas automatizadas, CI y contenedores) en equipos de 3 con roles diferenciados. AuraMap encaja perfecto:

- **Dominio simple de modelar** (batalla, ubicación, inscripción) que no se come el tiempo que necesitamos para la parte de DevOps.
- **Reglas de negocio que sí se pueden probar:** fecha futura, cupo entre 1 y 100, coordenadas válidas, distancia por fórmula de Haversine, cupo lleno y sin inscripciones duplicadas. Eso nos dio 15 pruebas con sentido y no pruebas de relleno.
- **Necesita base de datos de verdad** (PostgreSQL), lo que justifica `docker compose` con dos servicios, red privada, secreto y healthcheck.
- **Alcance acotado a propósito:** sin login, sin pagos, sin integraciones externas más allá del mapa de OpenStreetMap por CDN.
- **Nos motiva:** es un tema que nuestros propios compañeros entienden al toque, y eso hace que la demo se defienda sola.

c) Aplicación de Git con Conventional Commits

Qué son y por qué se usan: [Conventional Commits](#) es una convención para escribir mensajes de commit con el formato `tipo(alcance opcional): descripción en modo imperativo`. Se usa porque hace el historial legible y homogéneo entre los 3 integrantes, permite saber de un vistazo qué cambió y por qué (una feature, un fix, un cambio de infraestructura...) sin abrir el diff, y es la base para automatizar changelogs o versionado semántico si el proyecto creciera.

Tipos principales usados en AuraMap:

Tipo	Cuándo se usa	Ejemplo en este repo
<code>feat</code>	Funcionalidad nueva de producto	<code>feat(battles): publicar y listar batallas de aura</code>
<code>fix</code>	Corrección de un bug	<code>fix(db): recuperar conexiones tras reinicio de Postgres</code>
<code>docs</code>	Cambios de documentación	<code>docs: documentar prerrequisitos, uso de devops.sh y borrador del informe</code>
<code>test</code>	Pruebas nuevas o modificadas	<code>test(battles): pruebas de validación y endpoints de batallas</code>
<code>chore</code>	Tareas de mantenimiento/infra que no son feat ni fix	<code>chore(docker): Dockerfile multi-stage con Gunicorn y usuario no root</code>
<code>ci</code>	Cambios en integración continua	<code>ci: ejecutar pruebas y build de imagen en cada pull request</code>
<code>refactor</code>	Reestructurar código sin cambiar comportamiento	<code>refactor(battles): separar consultas a la DB de la lógica de dominio</code>

Regla del equipo (documentada en [CLAUDE.md](#)): un commit por ID del roadmap, mensaje exacto acordado de antemano, sin mezclar dos cambios en un commit ni partir uno en varios.

El historial en números: 19 commits de trabajo (35 contando los merge commits), 16 Pull Requests mergeados y 2 releases etiquetados en `main` : `v1.0.0` (cierre del roadmap) y `v1.1.0` (interfaz rediseñada).



```
bash - git log --graph
aaron@utec:~/aura-map$ git log --oneline --graph origin/main
* cfdb19a Merge pull request #15 from AaronCoorahua/release/1.1.0
| \
| * e5f18c9 chore(release): versión 1.1.0
| * b6078a3 Merge pull request #14 from AaronCoorahua/feature/rediseño-ui
| \
| | * 078159a docs(informe): actualizar capturas con la nueva interfaz
| | * 6ce0909 feat(ui): rediseñar la web con búsqueda por distrito, pin en el mapa e inscripción
| | \
| | * fffdeeb Merge pull request #13 from AaronCoorahua/main
| | \
| | * f09a7u3 Merge pull request #12 from AaronCoorahua/release/1.0.0
| | \
| | * 2dbad2d Merge branch 'develop' into release/1.0.0
| | \
| | * 0747331 Merge pull request #11 from AaronCoorahua/docs/capturas-informe
| | \
| | * f7d74c2 docs(informe): agregar capturas del proyecto corriendo y contexto del tema
| | \
| | * b02c1fb chore(release): versión 1.0.0
| | \
| | * 3c41135 Merge pull request #10 from AaronCoorahua/fix/tablas-y-seed
| | \
| | * cefdb0c fix(db): crear tablas al arrancar y agregar comando flask seed
| | * 32af181 Merge pull request #9 from AaronCoorahua/docs/readme-informe
| | \
| | \
| | * c32a0f3 docs: documentar prerrequisitos, uso de devops.sh y borrador del informe
| | \
| | * e65086d Merge pull request #8 from AaronCoorahua/fix/conexion-db
| | \
| | * 123289b fix(db): recuperar conexiones tras reinicio de Postgres
| | \
| | * b413e63 Merge pull request #7 from AaronCoorahua/ci/pruebas-en-pr
| | \
| | * 66800f5 ci: ejecutar pruebas y build de imagen en cada pull request
| | * 392c5a0 Merge pull request #6 from AaronCoorahua/feature/automatizacion
| | \
| | \
| | * 1dcbb38 chore(scripts): automatizar clonación, pruebas y ejecución local
| | * d2fe592 Merge pull request #5 from AaronCoorahua/feature/dockerizar
| | \
| | * 4bc9f73 chore(compose): orquestar flask y postgres con redes, secretos, volumen y healthcheck
| | * 9dc6f0b chore(docker): Dockerfile multi-stage con Gunicorn y usuario no root
| | \
| | * 5545fa2 Merge pull request #4 from AaronCoorahua/feature/inscripcion
| | \
| | * 066d2d4 feat(rsvp): inscripción a batallas con control de cupo
| | * cb4fb5e Merge pull request #3 from AaronCoorahua/refactor/capa-datos
| | \
| | * 360634d refactor(battles): separar consultas a la DB de la lógica de dominio
| | \
| | * c15a579 Merge pull request #2 from AaronCoorahua/feature/busqueda-cercana
| | \
| | * 5c1ed0d test(geo): pruebas de distancia y filtro por radio
| | * fd59fe0 feat(geo): buscar batallas cercanas por radio y mostrarlas en mapa
| | \
| | * 3189c99 Merge pull request #1 from AaronCoorahua/feature/publicar-batallas
| | \
| | * ab71d8e test(battles): pruebas de validación y endpoints de batallas
| | * 35c1546 feat(battles): publicar y listar batallas de aura
| | \
| | * 7fadf53 chore: inicializar proyecto AuraMap con estructura base
```

Figura 2. Historial con git log --oneline --graph: cada rama entra por merge commit y los releases llevan tag

Análisis de 6 commits (de un historial con más de 9)

- 35c1546 — feat(battles): publicar y listar batallas de aura** Introduce el modelo `Battle`, la validación de negocio en `app/services/battles.py` (título obligatorio, fecha futura, cupo entre 1 y 100, lat/Ing en rango) y los endpoints `POST /api/battles` y `GET /api/battles`. 4 archivos, 140 líneas agregadas. Es la primera feature de producto real del proyecto: define el modelo de datos que usan todos los commits siguientes (geo, rsvp, refactor de repositorios).
- ab71d8e — test(battles): pruebas de validación y endpoints de batallas** Agrega 6 pruebas unitarias sobre lo anterior: payload válido, título vacío, fecha pasada, cupo inválido, `POST` devuelve 201, `GET` lista. Un solo archivo (`tests/test_battles.py`), 59 líneas. Ejemplo de la disciplina del roadmap: cada feature va acompañada de su commit de pruebas por separado, nunca mezclados.
- 360634d — refactor(battles): separar consultas a la DB de la lógica de dominio** Mueve las consultas SQLAlchemy de `app/routes.py` a `app/repositories/battles.py`, dejando las rutas delgadas (solo orquestan validación + repositorio + respuesta HTTP). 3 archivos, 34 líneas agregadas, 12 eliminadas. Es un refactor puro: ninguna prueba existente se modificó, lo que confirma que el comportamiento externo no cambió.

4. **4bc9f73** — **chore(compose): orquestar flask y postgres con redes, secretos, volumen y healthcheck** Agrega `compose.yaml` y `.env.dev.example`. Configura dos redes (`public` y `private`, esta última `internal: true` para que Postgres no sea alcanzable desde fuera), un secreto Docker (`pg_password`) en vez de una variable de entorno en texto plano, un volumen (`postgres-data`) y un healthcheck con `pg_isready` del que depende el servicio `flask` (`condition: service_healthy`). 2 archivos, 56 líneas. Es el commit que hace reproducible el entorno completo con un solo comando.
5. **66809f5** — **ci: ejecutar pruebas y build de imagen en cada pull request** Agrega `.github/workflows/tests.yml`: en cada Pull Request hacia `develop` o `main`, instala Python 3.12, corre `./devops.sh test` (con fallback a `pytest` directo si el script no está) y hace `docker build` de la imagen. 1 archivo, 34 líneas. Cierra el ciclo: ya no depende de que cada persona recuerde correr las pruebas antes de pedir revisión.
6. **123289b** — **fix(db): recuperar conexiones tras reinicio de Postgres** Corrige un bug reproducido deliberadamente: con los servicios arriba, `docker compose restart postgres` seguido de peticiones a `/api/battles` producía `OperationalError: server closed the connection unexpectedly` en ~10% de los intentos, porque SQLAlchemy reutilizaba conexiones del pool que Postgres ya había cerrado. El fix agrega `pool_pre_ping: True` (SQLAlchemy verifica la conexión antes de reutilizarla) y una espera con reintentos y backoff exponencial al arrancar la app (`app/db.py::wait_for_db`), porque `depends_on: condition: service_healthy` solo ordena el arranque de los contenedores y no protege si Postgres cae después. 5 archivos,

incluyendo la evidencia de antes/después en [docs/evidencias/](#). Ejemplo de un fix con reproducción documentada antes de escribir la corrección.

```
bash - git show --stat (1/2)

aaron@utec:~/aura-map$ git show --stat --format='%h %s%n%an · %ad' --date=short 35c1546
35c1546 feat(battles): publicar y listar batallas de aura
Leonardo Candio · 2026-09-23

app/__init__.py      |  2 ++
app/models.py        | 34 ++++++
app/routes.py        | 36 ++++++
app/services/battles.py | 68 ++++++
4 files changed, 140 insertions(+)

aaron@utec:~/aura-map$ git show --stat --format='%h %s%n%an · %ad' --date=short ab71d8e
ab71d8e test(battles): pruebas de validación y endpoints de batallas
Leonardo Candio · 2026-09-23

tests/test_battles.py | 59 ++++++
1 file changed, 59 insertions(+)

aaron@utec:~/aura-map$ git show --stat --format='%h %s%n%an · %ad' --date=short 360634d
360634d refactor(battles): separar consultas a la DB de la lógica de dominio
Leonardo Candio · 2026-09-23

app/repositories/__init__.py |  1 +
app/repositories/battles.py  | 29 ++++++
app/routes.py                | 16 ++++-----
3 files changed, 34 insertions(+), 12 deletions(-)
```

Figura 3. git show --stat de los commits 1 a 3

```
bash - git show --stat (2/2)

aaron@utec:~/aura-map$ git show --stat --format='%h %s%n%an · %ad' --date=short 4bc9f73
4bc9f73 chore(compose): orquestar flask y postgres con redes, secretos, volumen y healthcheck
AaronCoorahua · 2026-09-23

.env.dev.example | 4 +++
compose.yaml      | 52 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
2 files changed, 56 insertions(+)

aaron@utec:~/aura-map$ git show --stat --format='%h %s%n%an · %ad' --date=short 66809f5
66809f5 ci: ejecutar pruebas y build de imagen en cada pull request
Leonardo Candio · 2026-09-23

.github/workflows/tests.yml | 34 +++++++++++++++++++++++++++++++++++++
1 file changed, 34 insertions(+)

aaron@utec:~/aura-map$ git show --stat --format='%h %s%n%an · %ad' --date=short 123289b
123289b fix(db): recuperar conexiones tras reinicio de Postgres
jjosnayo2102 · 2026-09-23

app/__init__.py      | 3 ++-
app/config.py         | 2 ++
app/db.py             | 25 +++++++++++++++++++++++++++++++++++++
docs/evidencias/c10-antes.txt | 22 +++++++++++++++++++++++++++++++++++++
docs/evidencias/c10-despues.txt | 21 +++++++++++++++++++++++++++++++++++++
5 files changed, 72 insertions(+), 1 deletion(-)
```

Figura 4. git show --stat de los commits 4 a 6

Pull Requests según la metodología de branching: el equipo usa GitFlow — rama `develop` como integración, `main` como estable, y una rama por bloque del roadmap (`feature/...` , `fix/...` , `chore/...` , `docs/...` , `release/...`) que se integra a `develop` (o a `main` en el release) por Pull Request con merge commit (`--no-ff` , sin squash ni rebase), siempre con al menos un revisor asignado según la tabla de roles del roadmap. Cada release sigue el ciclo completo de GitFlow: rama `release/x.y.z` desde `develop` → PR a `main` → tag `vx.y.z` → PR de

`main` de vuelta a `develop`. El CI (`.github/workflows/tests.yml`) corre las pruebas y construye la imagen Docker en cada PR, y ninguno se mergeó con el CI en rojo.

All pull requests		New pull request	
is:pr is:merged			
Open	0	Closed	16
Author ▾ Label ▾ Projects ▾ Milestones ▾ Reviews ▾ Assignee ▾ ⌵ Newest ▾			
🔖 chore(release): integrar main 1.1.0 de vuelta a develop			
#16 · AaronCoorahua opened 2 days ago · Closed 2 days ago · Owner · Review required · ✓ 1/1			
🔖 chore(release): versión 1.1.0			
#15 · AaronCoorahua opened 2 days ago · Closed 2 days ago · Owner · Review required · ✓ 1/1			
🔖 feat(ui): rediseñar la web con búsqueda por distrito, pin en el mapa e inscripción			
#14 · AaronCoorahua opened 2 days ago · Closed 2 days ago · Owner · Review required · ✓ 1/1			
🔖 chore(release): integrar main 1.0.0 de vuelta a develop			
#13 · AaronCoorahua opened 2 days ago · Closed 2 days ago · Owner · Review required · ✓ 1/1			
🔖 chore(release): versión 1.0.0			
#12 · josnayo2102 opened 2 days ago · Closed 2 days ago · Collaborator · Approved · ✓ 1/1			
🔖 docs(informe): agregar capturas del proyecto corriendo y contexto del tema			
#11 · AaronCoorahua opened 2 days ago · Closed 2 days ago · Owner · Review required · ✓ 1/1			
🔖 fix(db): crear tablas al arrancar y agregar comando flask seed			
#10 · AaronCoorahua opened 2 days ago · Closed 2 days ago · Owner · Review required · ✓ 1/1			
🔖 docs: documentar prerequisites, uso de devops.sh y borrador del informe			
#9 · josnayo2102 opened 2 days ago · Closed 2 days ago · Collaborator · Approved · ✓ 1/1			
🔖 fix(db): recuperar conexiones tras reinicio de Postgres			
#8 · josnayo2102 opened 2 days ago · Closed 2 days ago · Collaborator · Approved · ✓ 1/1			
🔖 ci: ejecutar pruebas y build de imagen en cada pull request			
#7 · kindyy opened 3 days ago · Closed 2 days ago · Collaborator · Review required · ✓ 1/1			
🔖 chore(scripts): automatizar clonación, pruebas y ejecución local			
#6 · AaronCoorahua opened 3 days ago · Closed 2 days ago · Owner · Review required			
🔖 chore(docker): Dockerfile multi-stage y orquestación con compose			
#5 · AaronCoorahua opened 3 days ago · Closed 3 days ago · Owner · Approved			
🔖 feat(rsvp): inscripción a batallas con control de cupo			
#4 · AaronCoorahua opened 3 days ago · Closed 3 days ago · Owner · Approved			
🔖 refactor(battles): separar consultas a la DB de la lógica de dominio			
#3 · kindyy opened 3 days ago · Closed 3 days ago · Collaborator · Approved			
🔖 feat(geo): buscar batallas cercanas por radio y mostrarlas en mapa			
#2 · AaronCoorahua opened 3 days ago · Closed 3 days ago · Owner · Approved			
🔖 feat(battles): publicar y listar batallas de aura			
#1 · kindyy opened 3 days ago · Closed 3 days ago · Collaborator · Approved			

Figura 5. Pull Requests del proyecto en GitHub

d) Pruebas unitarias

Herramientas: `pytest` como framework de pruebas y `pytest-cov` para el reporte de cobertura. Las pruebas usan el test client de Flask y una base de datos SQLite en memoria (fixture `app` en `tests/conftest.py`), por lo que no dependen de Docker ni de Postgres para correr.

Implementación: 15 pruebas unitarias repartidas en 4 archivos (mínimo exigido: 9):

Archivo	Pruebas	Qué cubre
<code>tests/test_api.py</code>	2	<code>/health</code> responde 200 e incluye la versión.
<code>tests/test_battles.py</code>	6	Validación de payload (título vacío, fecha pasada, cupo inválido) y endpoints <code>POST</code> / <code>GET</code> de batallas.
<code>tests/test_geo.py</code>	4	<code>haversine_km</code> (mismo punto = 0, 1° de longitud en el ecuador ≈ 111.19 km), filtro por radio y coordenadas inválidas.
<code>tests/test_rsvp.py</code>	3	Inscripción con cupo disponible, rechazo por cupo lleno (409) y rechazo por duplicado (409).

Evidencia de ejecución:

```
./devops.sh test
# equivalente: pytest -v --cov=app --cov-report=term-missing

15 passed in 0.79s
TOTAL cobertura: 79%
```

```
bash - ./devops.sh test

aaron@utec:~/aura-map$ ./devops.sh test

==> Instalando dependencias de desarrollo

==> Ejecutando las pruebas
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.2.2, pluggy-1.6.0 -- /home/aaron_coorahua/aura-map/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/aaron_coorahua/aura-map
plugins: cov-5.0.0
collecting ... collected 15 items

tests/test_api.py::test_health_responde_ok PASSED [ 6%]
tests/test_api.py::test_health_incluye_version PASSED [ 13%]
tests/test_battles.py::test_payload_valido PASSED [ 20%]
tests/test_battles.py::test_titulo_vacio PASSED [ 26%]
tests/test_battles.py::test_fecha_pasada PASSED [ 33%]
tests/test_battles.py::test_cupo_invalido PASSED [ 40%]
tests/test_battles.py::test_post_devuelve_201 PASSED [ 46%]
tests/test_battles.py::test_get_lista_batallas PASSED [ 53%]
tests/test_geo.py::test_mismo_punto_da_cero PASSED [ 60%]
tests/test_geo.py::test_un_grado_de_longitud_en_el_ecuador PASSED [ 66%]
tests/test_geo.py::test_el_filtro_excluye_las_lejanas PASSED [ 73%]
tests/test_geo.py::test_coordenadas_invalidas_lanzan_value_error PASSED [ 80%]
tests/test_rsvp.py::test_inscribe_cuando_hay_cupo PASSED [ 86%]
tests/test_rsvp.py::test_rechaza_cuando_no_hay_cupo PASSED [ 93%]
tests/test_rsvp.py::test_rechaza_inscripcion_duplicada PASSED [100%]

----- coverage: platform linux, python 3.12.3-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
app/__init__.py                     20      2    90%   22-23
app/config.py                       18      6    67%   13, 19-23
app/db.py                           24      7    71%   25-29, 37-39
app/models.py                       23      0   100%
app/repositories/__init__.py         0      0   100%
app/repositories/battles.py         16      1    94%    20
app/repositories/rsvp.py             11      0   100%
app/routes.py                       53     16    70%   24-25, 39-51, 58-61, 68, 72-73, 86
app/seed.py                         13      8    38%   9, 51-57
app/services/__init__.py             0      0   100%
app/services/battles.py             46     10    78%   13, 21, 24-25, 27, 40, 42, 50, 55, 58
app/services/geo.py                 31      6    81%   15-16, 21, 46-47, 49
app/services/rsvp.py                 19      2    89%   24, 28
-----
TOTAL                               274     58    79%

===== 15 passed in 0.79s =====
```

Figura 6. ./devops.sh test: 15 pruebas en verde y reporte de cobertura

La cobertura bajó de 82% a 79% al integrar `app/seed.py` (el comando `flask seed`), que no tiene prueba propia: es el siguiente candidato a cubrir. Las pruebas corren en menos de un segundo porque la lógica de negocio vive en funciones puras (`app/services/`) y la DB de prueba es SQLite en memoria: se pueden correr en cada guardado, no solo antes de un PR.

Las mismas reglas, probadas contra la app real. Además de pytest, verificamos a mano con `curl` las reglas de inscripción sobre la app corriendo en Docker con Postgres: una batalla con cupo 2 acepta a Camila y a Yhoel (201), rechaza a Mateo porque ya no hay cupo (409) y rechaza

que Camila se inscriba dos veces en la misma batalla (409). La búsqueda por radio de 2 km desde el Parque Kennedy devuelve solo las batallas de Miraflores:

```
bash - API de inscripción

aaron@utec:~/aura-map$ curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/battles -H 'Content-Type: application/json' -d @batalla.json
{"cupo":2,"descripcion":"Solo 2 cupos: el que parpadea primero pierde.","direccion":"Parque Reducto N.\u00b02","distrito":"Miraflores","fecha":"2026-10-15T20:00:00","id":4,"lat":-12.1287,"lng":-77.0263,"organizador":"Equipo AuraMap","tiene_permiso":true,"titulo":"Mano a mano de aura en el Parque Reducto"}

HTTP 201

aaron@utec:~/aura-map$ curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/battles/4/rsvp -H 'Content-Type: application/json' -d '{"nombre": "Camila","battle_id":4,"id":1,"nombre":"Camila"}'

HTTP 201

aaron@utec:~/aura-map$ curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/battles/4/rsvp -H 'Content-Type: application/json' -d '{"nombre": "Yhoel","battle_id":4,"id":2,"nombre":"Yhoel"}'

HTTP 201

aaron@utec:~/aura-map$ curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/battles/4/rsvp -H 'Content-Type: application/json' -d '{"nombre": "Mano a mano de aura en el Parque Reducto","battle_id":4,"id":3,"nombre":"Mano a mano de aura en el Parque Reducto"}'

{"error":"La batalla ya no tiene cupo disponible."}

HTTP 409

aaron@utec:~/aura-map$ curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/battles/1/rsvp -H 'Content-Type: application/json' -d '{"nombre": "Camila","battle_id":1,"id":3,"nombre":"Camila"}'

HTTP 201

aaron@utec:~/aura-map$ curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/battles/1/rsvp -H 'Content-Type: application/json' -d '{"nombre": "Camila","battle_id":1,"id":4,"nombre":"Camila"}'

{"error":"Esa persona ya est\u00e1 inscrita en la batalla."}

HTTP 409

aaron@utec:~/aura-map$ curl -s 'localhost:8080/api/battles?lat=-12.1211&lng=-77.0297&radius_km=2' | python3 -c "import json,sys; [print(b['id'], '|', b['titulo'], '|', b['direccion'], '|', b['distrito']) for b in json.loads(sys.stdin.read())]"

1 | Batalla de Aura en el Parque Kennedy | Miraflores
4 | Mano a mano de aura en el Parque Reducto | Miraflores
```

Figura 7. Inscripciones por API: 201 con cupo, 409 por cupo lleno y 409 por duplicado

e) Automatización en scripts bash

Todo el ciclo de desarrollo se automatiza con `devops.sh` (bash, `set -euo pipefail`, ayuda con `-h`, compatible con Linux, macOS y Git Bash/WSL en Windows):

- **Clonación:** `./devops.sh clone <url> [dir]` clona el repositorio, crea el entorno virtual e instala `requirements-dev.txt`.
- **Pruebas:** `./devops.sh test` crea el entorno virtual si falta y corre `pytest -v --cov=app --cov-report=term-missing`.
- **Modo local:** `./devops.sh local` levanta la app sin Docker, con SQLite en `instance/`, carga 3 batallas de ejemplo con `flask seed` y arranca `flask --app app:create_app run --debug`.

Además, `./devops.sh docker` (genera secretos y levanta Flask + Postgres), `./devops.sh down [-v]` (baja los servicios) y `./devops.sh evidence` (guarda las evidencias de este informe en `docs/evidencias/`).

La secuencia completa, tal como la haría un integrante nuevo desde cero (Ubuntu en WSL, sin nada instalado del proyecto):

1. Clonar — clona el repo, crea el entorno virtual e instala dependencias:

A terminal window with a dark background and light green text. The title bar shows three colored circles (red, yellow, green) and the text 'bash - ./devops.sh clone'. The prompt is 'aaron@utec:~\$'. The command entered is './devops.sh clone https://github.com/AaronCoorahua/aura-map.git'. The output shows the script cloning the repository, creating a virtual environment in .venv, installing dependencies, and finally listing the next steps: 'cd aura-map && ./devops.sh test'.

```
bash - ./devops.sh clone

aaron@utec:~$ ./devops.sh clone https://github.com/AaronCoorahua/aura-map.git

==> Clonando https://github.com/AaronCoorahua/aura-map.git en aura-map
Cloning into 'aura-map'...

==> Creando el entorno virtual en .venv

==> Instalando dependencias de desarrollo

==> Listo. Entra con: cd aura-map && ./devops.sh test
```

Figura 8. `./devops.sh clone`

2. Probar — la misma salida de la sección d) (15 passed).

3. Correr en local — sin Docker, con SQLite y datos de ejemplo:

```
bash - ./devops.sh local

aaron@utec:~/aura-map$ ./devops.sh local

==> Instalando dependencias de desarrollo

==> Cargando datos de ejemplo
Batallas de ejemplo insertadas: 3

==> Arrancando en http://localhost:5000 (Ctrl+C para cortar)
* Serving Flask app 'app:create_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 973-685-329
127.0.0.1 - - [23/Sep/2026 12:35:01] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [23/Sep/2026 12:35:01] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [23/Sep/2026 12:35:01] "GET /api/battles HTTP/1.1" 200 -
^C
```

Figura 9. ./devops.sh local: seed y servidor de desarrollo en el puerto 5000

Mientras corre, desde otra terminal la API ya responde con las 3 batallas sembradas (Miraflores, Cercado de Lima y Barranco):

```
bash - otra terminal

aaron@utec:~/aura-map$ curl -s localhost:5000/health
{
  "status": "ok",
  "version": "0.1.0"
}

aaron@utec:~/aura-map$ curl -s localhost:5000/api/battles | python3 -c "import json,sys; [print(b['id'], '|', b['titulo'], '|', b['distrito']) for b in json.loads(sys.stdin.read())]"
1 | Batalla de Aura en el Parque Kennedy | Miraflores
2 | Duelo de Aura en la Plaza San Martín | Cercado de Lima
3 | Aura Farming al atardecer en el Malecón de Barranco | Barranco
```

Figura 10. Consultas a la app en modo local

Con tres comandos alguien del equipo pasa de "no tengo nada" a "tengo la app corriendo con datos y las pruebas en verde", sin leer un README de dos páginas ni preguntar por WhatsApp qué versión de Python instalar.

f) Dockerizar su aplicación

Dockerfile: build multi-stage. La etapa `builder` (`python:3.12-slim`) instala las dependencias en un entorno virtual; la etapa final, también `python:3.12-slim` , copia solo ese entorno virtual y el código de la app, corre como usuario sin privilegios (`aura` , no root), expone el puerto 5000 y arranca con Gunicorn en forma exec: `CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", ..., "app:create_app()"]` .

Docker Compose — servicio web (Flask + Gunicorn):

```
flask:
  image: aura-map:latest
  build:
    context: .
    dockerfile: Dockerfile
  ports:
    - "8080:5000"
  env_file:
    - .env.dev
  environment:
    APP_VERSION: "1.1.0"
    DB_HOST: postgres
    DB_DATABASE: auradb
    DB_USER: aurauser
  networks:
    - public
    - private
  depends_on:
    postgres:
      condition: service_healthy
```

Docker Compose — base de datos (Postgres):

```
postgres:
  image: postgres:16.3
  environment:
    POSTGRES_USER: aurauser
    POSTGRES_DB: auradb
    POSTGRES_PASSWORD_FILE: /run/secrets/pg_password
  secrets:
    - pg_password
  volumes:
    - postgres-data:/var/lib/postgresql/data
  networks:
    - private
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U aurauser -d auradb"]
    interval: 5s
    timeout: 5s
    retries: 5
```

Postgres no publica puertos al host (solo es alcanzable desde la red `private`, que es `internal: true`); su contraseña se inyecta por Docker secret (`secrets/pg_password.txt`, ignorado por git), nunca en texto plano en `compose.yaml`.

Evidencia de la aplicación corriendo en Docker. Con un solo comando se generan los secretos, se construye la imagen, se crean las redes y el volumen, se espera a que Postgres esté *healthy*, se prueba `/health` y se cargan las batallas de ejemplo:

```
bash - ./devops.sh docker

aaron@utec:~/aura-map$ ./devops.sh docker

==> Generando una contraseña nueva para Postgres

==> Levantando los servicios
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[... salida de docker build recortada ...]
flask Built
Network aura-map_public Creating
Network aura-map_public Created
Network aura-map_private Creating
Network aura-map_private Created
Volume "aura-map_postgres-data" Creating
Volume "aura-map_postgres-data" Created
Container aura-map-postgres-1 Creating
Container aura-map-postgres-1 Created
Container aura-map-flask-1 Creating
Container aura-map-flask-1 Created
Container aura-map-postgres-1 Starting
Container aura-map-postgres-1 Started
Container aura-map-postgres-1 Waiting
Container aura-map-postgres-1 Healthy
Container aura-map-flask-1 Starting
Container aura-map-flask-1 Started

==> Esperando a que Postgres esté healthy

==> Postgres healthy tras 1 intento(s)

==> Probando /health
curl: (56) Recv failure: Connection reset by peer
{"status":"ok","version":"0.1.0"}

==> Cargando datos de ejemplo
Batallas de ejemplo insertadas: 3

==> Estado de los servicios
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
aura-map-flask-1     aura-map:latest      "gunicorn --bind 0.0... flask        17 seconds ago Up 6 seconds         0.0.0.0:8080->5000/tcp
aura-map-postgres-1 postgres:16.3        "docker-entrypoint.s... postgres   17 seconds ago Up 15 seconds (healthy)
```

Figura 11. `./devops.sh docker`: build, redes, volumen, healthcheck, `/health` y seed

El `curl: (56) Recv failure` que aparece una vez no es un error: es `devops.sh` reintentando `/health` mientras Gunicorn termina de arrancar (Postgres *healthy* no significa que Flask ya esté escuchando). Al segundo intento responde `{"status":"ok"}`.

Verificación de aislamiento. `./devops.sh evidence` confirma que la red `private` es `internal=true` y que Postgres no publica el puerto 5432 al host: la base de datos solo es alcanzable desde el contenedor de Flask.

```
bash - ./devops.sh evidence

aaron@utec:~/aura-map$ ./devops.sh evidence

==> Guardando evidencias en docs/evidencias
c10-antes.txt
c10-despues.txt
compose-ps.txt
git-log.txt
health.txt
postgres-puerto.txt
pytest.txt
red-privada.txt

aaron@utec:~/aura-map$ cat docs/evidencias/red-privada.txt
internal=true
aaron@utec:~/aura-map$ cat docs/evidencias/postgres-puerto.txt
Sin puertos publicados: Postgres solo es alcanzable desde la red privada.
aaron@utec:~/aura-map$ docker compose port postgres 5432
no port 5432/tcp for container aura-map-postgres-1:

aaron@utec:~/aura-map$ docker compose ps
NAME                IMAGE            COMMAND                  SERVICE    CREATED        STATUS               PORTS
aura-map-flask-1    aura-map:latest  "gunicorn --bind 0.0..." flask      20 seconds ago Up 9 seconds        0.0.0.0:8080->5000/tcp
aura-map-postgres-1 postgres:16.3     "docker-entrypoint.s..." postgres   20 seconds ago Up 18 seconds (healthy)
```

Figura 12. Red privada interna y Postgres sin puertos publicados

La app en el navegador (<http://localhost:8080>). Nadie busca un evento escribiendo latitud y longitud, así que la interfaz habla en el idioma de Lima: a la izquierda las batallas como tarjetas (fecha, lugar, organizador, permiso municipal y una barra con los cupos que quedan) y a la derecha el mapa. Al tocar una tarjeta el mapa vuela al pin y abre su ficha:

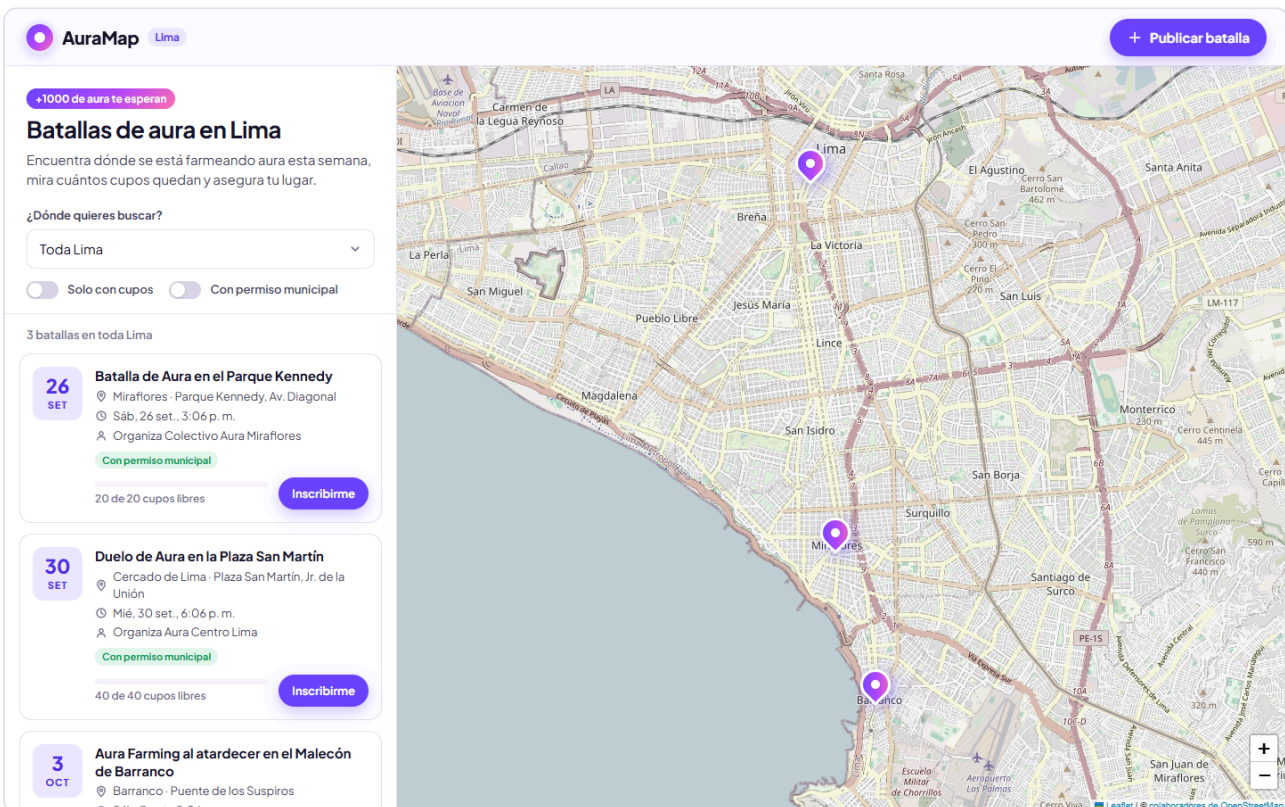


Figura 13. Mapa de AuraMap con las batallas de Lima

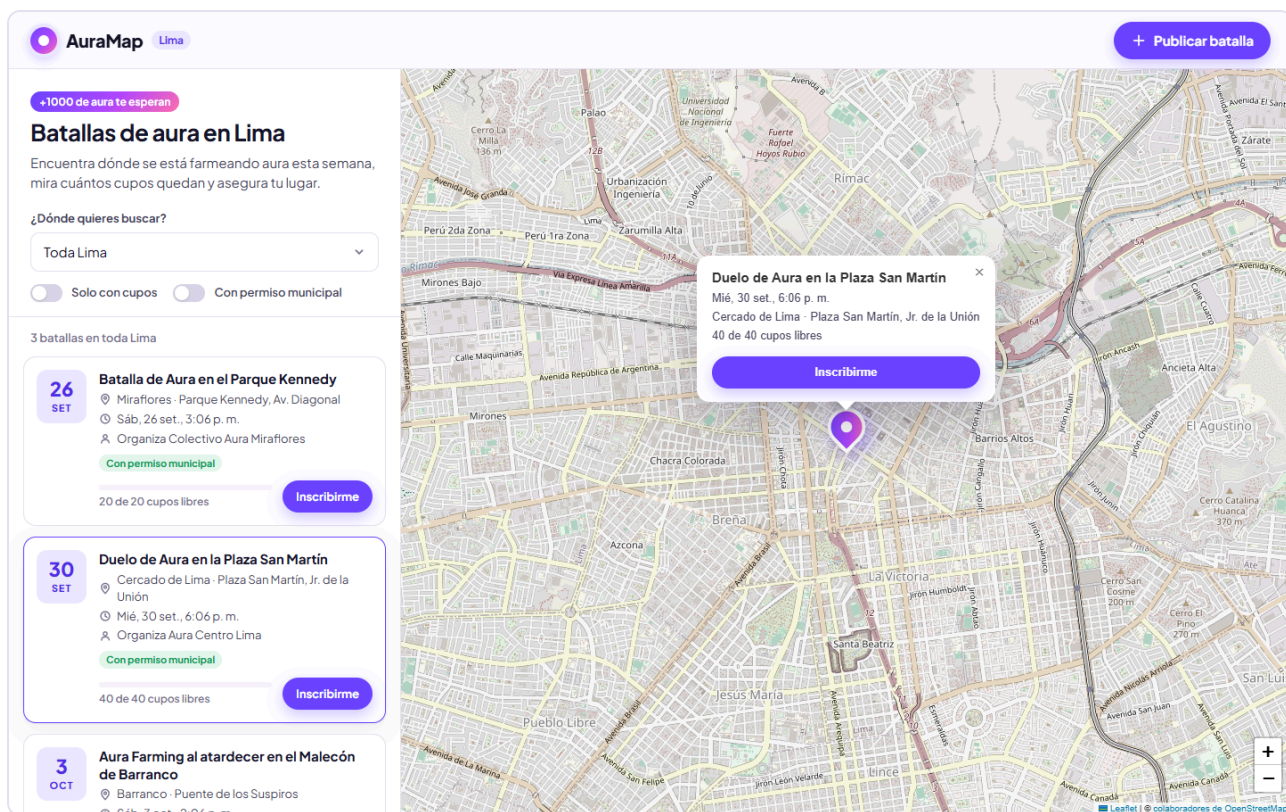


Figura 14. Tarjeta seleccionada y ficha de la batalla en el mapa

Buscar como humano. En "¿Dónde quieres buscar?" se elige un distrito de Lima o "Cerca de mí" (usa la ubicación del celular) y una distancia máxima en chips (2, 5, 10 o 20 km). El mapa dibuja

el radio y cada tarjeta muestra a cuántos km queda. Aquí, batallas a menos de 2 km de Miraflores:

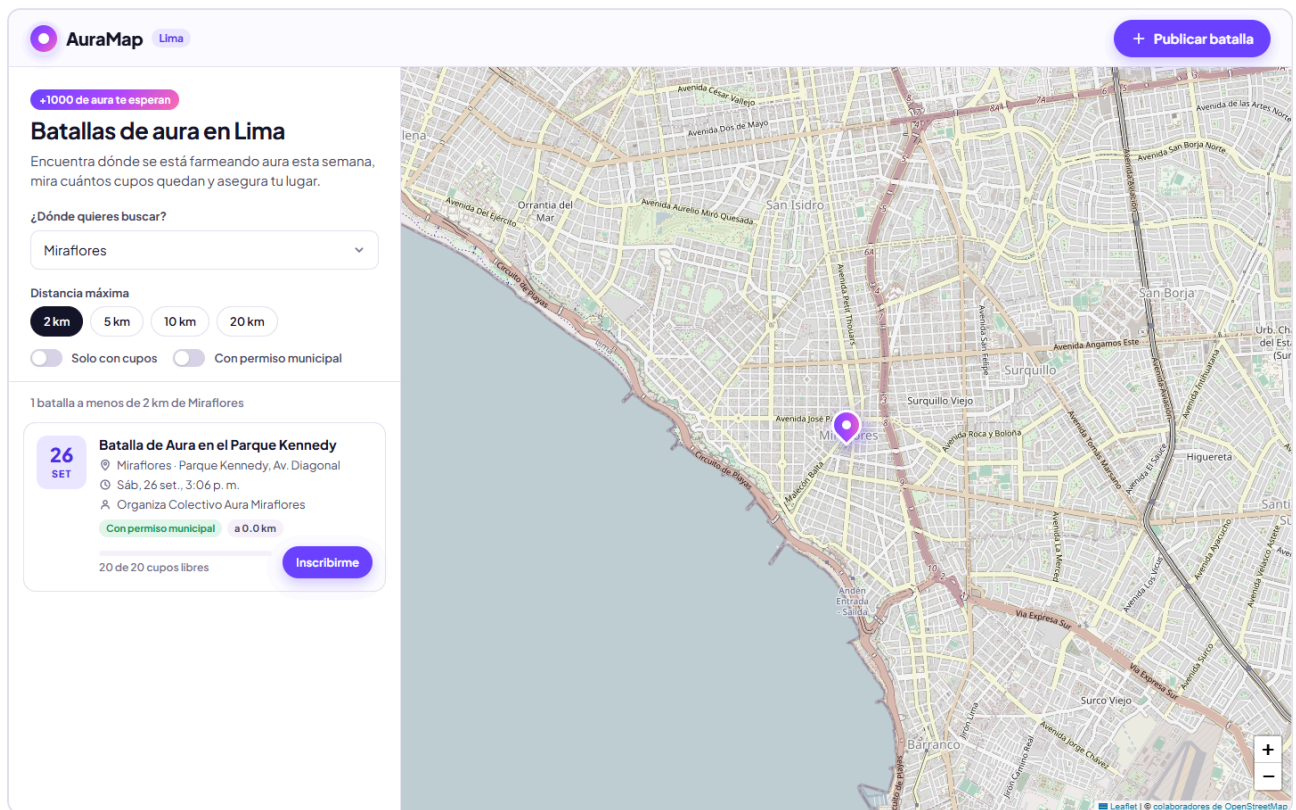


Figura 15. Búsqueda por distrito y distancia

Inscribirse en dos toques. El botón "Inscribirme" pide solo un nombre; si la batalla se llenó o esa persona ya está inscrita, el mismo diálogo muestra el error de la API (409) en lenguaje humano:

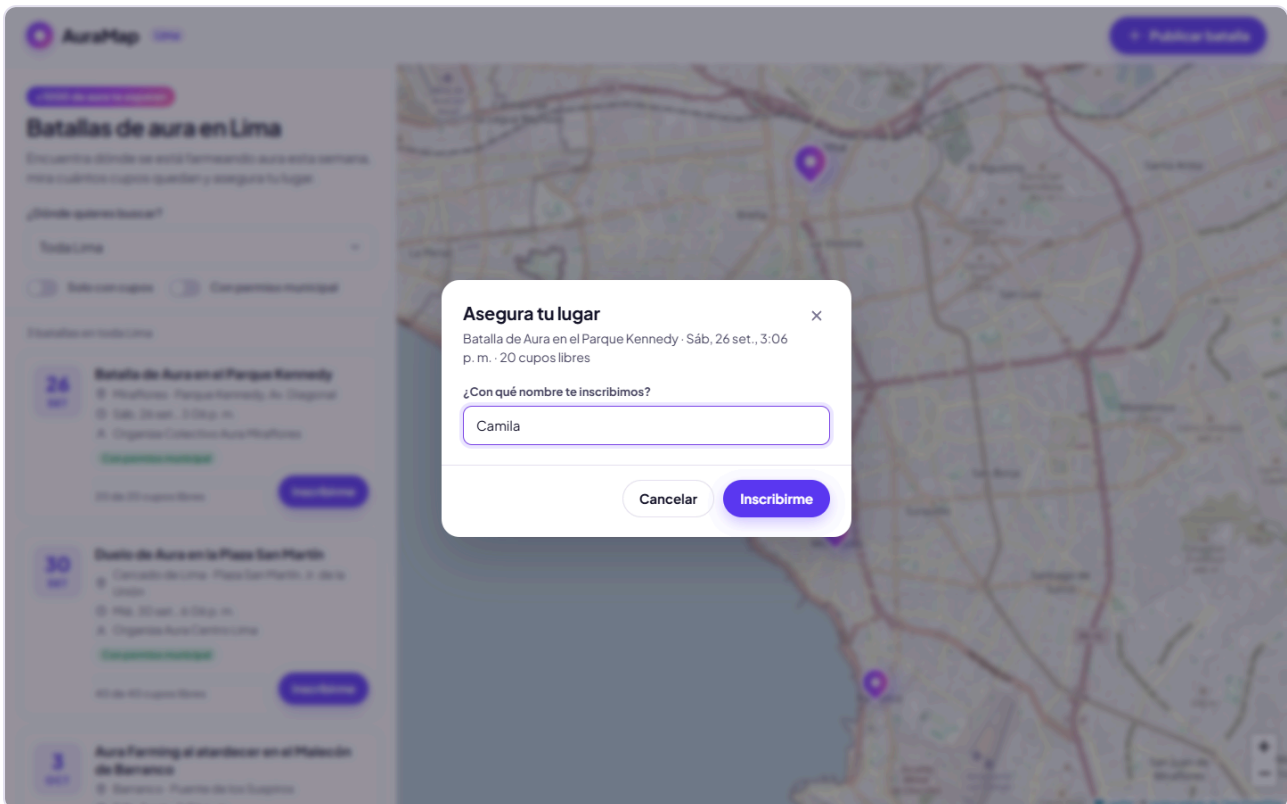


Figura 16. Diálogo de inscripción

Publicar sin coordenadas. El organizador elige el distrito, el minimapa se centra ahí y basta con tocar o arrastrar el pin hasta el lugar exacto. Las reglas se validan en el navegador y otra vez en

el backend (fecha futura, cupo 1–100, coordenadas válidas); al publicar, la batalla aparece al instante en el mapa, guardada en Postgres:

Publica tu batalla

Cuéntale a Lima dónde y cuándo se mide el aura.

Nombre de la batalla
Batalla de Aura UTEC: +1000 aura o nada

Distrito
Barranco

Descripción opcional
Cachimbos vs. ciclo 10. Gana quien mantenga la calma en semana de parciales.

Dirección o referencia
Jr. Medrano Silva 165, UTEC

Ubicación exacta
Arrastra el pin o toca el mapa para ajustar el lugar exacto.

Fecha
09/24/2026

Hora
06:30 PM

Cupos
30

Organiza
Equipo AuraMap

☒ Tengo permiso municipal

Cancelar **Publicar batalla**

Figura 17. Formulario de publicación con pin en el mapa

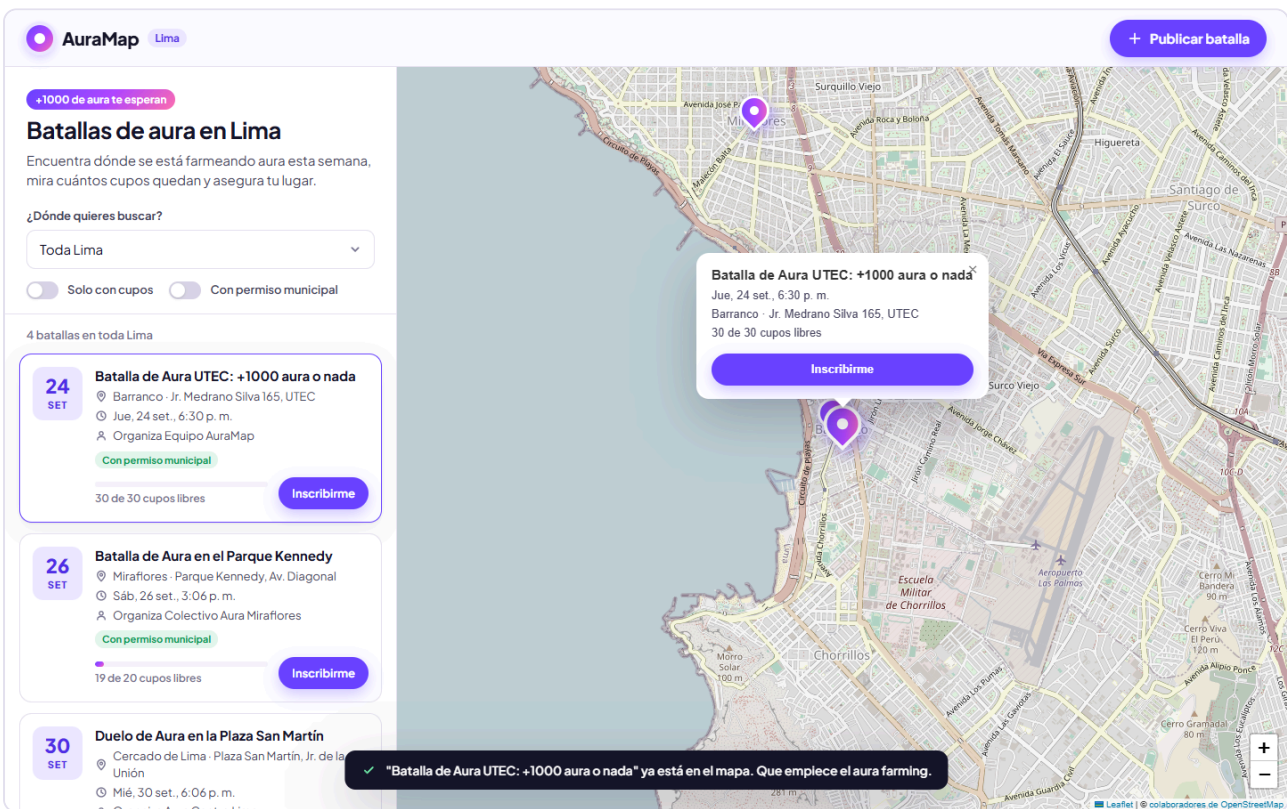


Figura 18. Batalla publicada y seleccionada en el mapa

Pensada para celular, que es donde se comparten estas batallas: el mapa pasa arriba y la lista abajo, sin scroll horizontal.

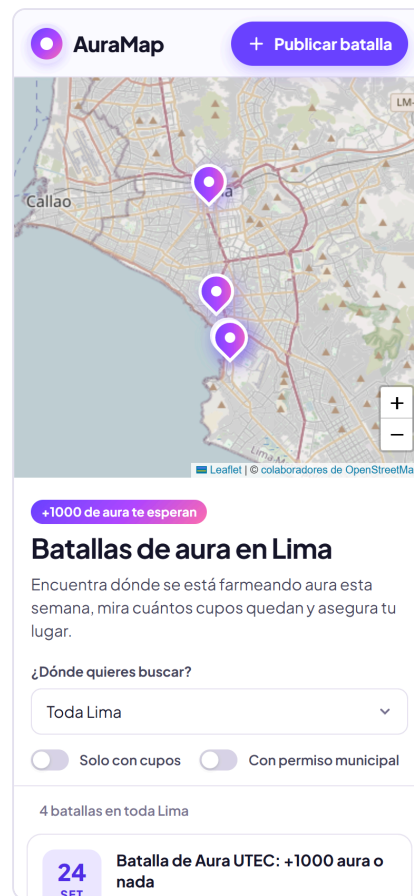


Figura 19. Versión móvil

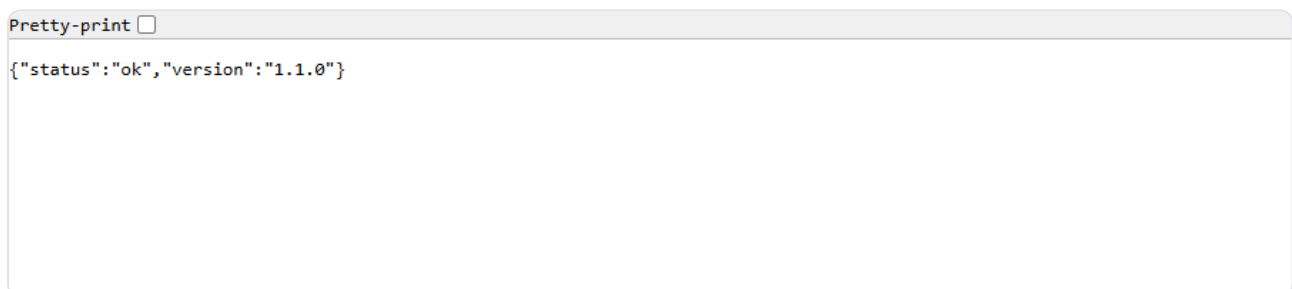


Figura 20. /health en el navegador

Un hallazgo que solo apareció al dockerizar. En la primera corrida con Postgres, `/api/battles` respondía 500 (`relation "battle" does not exist`): las pruebas usan SQLite en memoria y crean las tablas en el fixture, así que nunca vieron el problema. Lo detectó C al reproducir el bug de `fix(db)` y se corrigió en `fix(db): crear tablas al arrancar y agregar comando flask seed` (PR #10): `create_app` crea las tablas que faltan (tolerando que los 2 workers de Gunicorn lo intenten a la vez) y `flask seed` carga los datos de ejemplo.

g) Conclusiones preliminares

- **Git con convenciones claras:** usar Conventional Commits desde el primer commit y un roadmap con un ID y un mensaje fijo por bloque eliminó la ambigüedad de "qué se supone que hace este commit" al revisar PRs entre 3 personas trabajando en paralelo. La trazabilidad se nota en la práctica: el bug de `fix(db)` (commit `123289b`) se pudo documentar y revisar sin necesidad de explicación extra, porque el tipo y el alcance del commit ya dicen qué se tocó y por qué.
- **Pruebas unitarias:** correr las 15 pruebas en menos de 2 segundos, sin Docker ni Postgres, permitió refactorizar (`360634d`) con la garantía de que el comportamiento externo no cambiaba, y detectar en CI cualquier regresión antes de mergear. La cobertura (79%) además señala qué ramas del código (sobre todo manejo de errores en rutas) todavía no están probadas.
- **Dockerizar la aplicación:** reproducir el entorno completo con un solo comando (`./devops.sh docker`) hizo posible reproducir de forma determinística un bug real (conexiones caídas tras reiniciar Postgres) en vez de solo poder discutirlo en teoría, y las redes/secrets/healthcheck de `compose.yaml` obligan a modelar desde el inicio cómo se comunican los servicios en producción, no solo en la laptop de cada integrante.
- **Probar en el entorno real, no solo en el de pruebas:** 15 pruebas en verde no impidieron que la app fallara con una base de datos nueva en Postgres. El entorno reproducible de Docker fue lo que hizo visible esa diferencia entre SQLite en memoria y producción; la lección para el final del curso es agregar una prueba de humo contra `docker compose` en el CI.
- **El producto también importa:** elegir un tema que el equipo y sus compañeros entienden al toque (el meme del aura) hizo que las reglas de negocio salieran naturales —cupos, duplicados, permiso municipal— y que la demo se explique sola: publicas tu batalla, la gente la encuentra en el mapa y se inscribe antes de que se llene.

Anexo: evidencias

- Capturas de pantalla: `imagenes/` (numeradas en el orden en que aparecen en este informe).
- Salidas de comandos en texto: `docs/evidencias/`, generadas con `./devops.sh evidence` (`git-log.txt`, `pytest.txt`, `compose-ps.txt`, `health.txt`, `red-privada.txt`, `postgres-puerto.txt`) más el antes/después del bug de conexiones (`c10-antes.txt`, `c10-despues.txt`).